CS3230 Lecture 07+**08**

Introduction to NP-completeness

Lecture Topics and Readings:

- Complexity classes [CLRS] Ch34.1-34.2
- Polynomial-time reduction [CLRS] Ch34.3
- **Circuit Satisfiability is NP-complete [CLRS] Ch34.3 (today)**
- **More NP-complete problems [CLRS] Ch34.5 (today)**

I cannot find an efficient algorithm,
but neither can all these famous people.

Quick Review (1)

A super-polynomial algorithm runs very [fast/slow]

Can *every* Optimization problems be converted into Decision problems? [Yes/No]

NP stands for [NoProblem/NonPolynomial/ Non-deterministicPolynomial]

Tautology, MST, Integer-factorization, and Subset-SUM belong to which class? Choice: [P/NP/co-NP]

- Tautology $\in$ co-NP, MST $\in$ P,
  Integer-factorization $\in$ NP and co-NP, Subset-Sum $\in$ NP

Quick Review (2)

Is this statement true: "There is no way a polynomial solution exists for Subset-SUM problem?" [Yes/No]

- Subset-SUM is unlikely to have polynomial solution unless P=NP

Mat_multi(plication) problem can be reduced to Mat_sqr(uare) problem in polynomial time? [Yes/No]

Suppose A $\leq_p$ B, what are the implications?

1. If B is solvable in polynomial time (easy), then A is [hard/easy]
2. If A is hard, then B is [hard/easy]

NP-complete (NPC) - Review

A problem **L** is **NP-hard** if any other problem **L'**$\in$**NP** can be reduced to **L** in polynomial time (i.e. **L'** $\leq_p$ **L**)

So if **L** can be solved in polynomial time, then **all problems in NP** can be solved in polynomial time :O

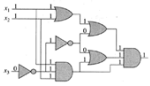
If **L** is **NP-hard** and **L**$\in$**NP**,
then **L**$\in$**NP-complete (NPC)**

Circuit Satisfiability (CIRCUIT-SAT) is NP-complete!

Proof of the 1st NPC
Our starting point…

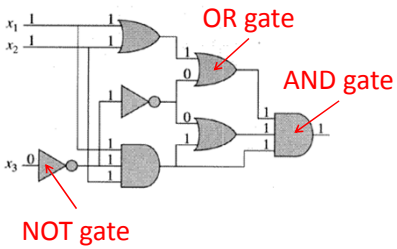
CIRCUIT-SAT (a 'wordy' definition)

Boolean combinational circuit



- Boolean combinational elements, wired together
- Each element has inputs and outputs (in binary)
- Limit the number of output to 1
- The elements are *logic gates*: NOT gate, AND gate, OR gate
- *truth table* gives the outputs for each setting of inputs of the logic gates
- truth assignment is a set of Boolean inputs
- satisfying assignment is a truth assignment that causes the output to be 1
- A circuit is satisfiable if it has a satisfying assignment

CIRCUIT-SAT Example



(x_1=1, x_2=1, x_3=0) is a satisfying assignment

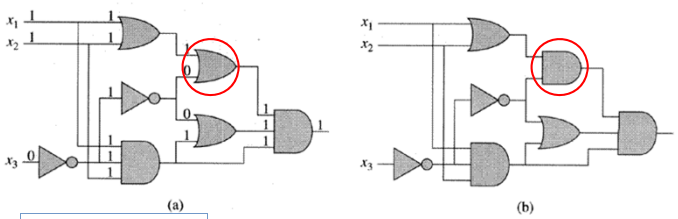
CIRCUIT-SAT

Input: A Boolean circuit composed of logic gates AND, OR, and NOT

Output: Does there exist an assignment of the variables so that the circuit is satisfiable?

Implication: In the area of computer-aided hardware optimization, if a sub-circuit always produces 0, then the sub-circuit can be replaced by a simpler sub-circuit that omits all gates and just output a 0

Two instances of CIRCUIT-SAT



What you saw earlier

Figure 34.8 Two instances of the circuit-satisfiability problem. **(a)** The assignment ($x_1 = 1$, $x_2 = 1$, $x_3 = 0$) to the inputs of this circuit causes the output of the circuit to be 1. The circuit is therefore satisfiable. **(b)** No assignment to the inputs of this circuit can cause the output of the circuit to be 1. The circuit is therefore unsatisfiable.

Solving CIRCUIT-SAT problem

Intuitive ☺ (but brute-force ☹) solution: For each possible assignment, we check whether it generates a 1

Suppose the number of inputs is k,
then the total possible assignments are 2^k

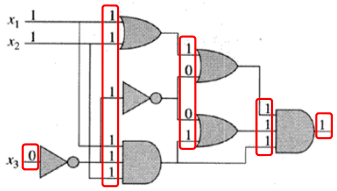
- The running time is $\Omega(2^k)$
- This is <u>NOT</u> a polynomial time solution ☹

CIRCUIT-SAT problem is in NP

Proof: CIRCUIT-SAT is polynomial time <u>verifiable</u>

Certificate: The assignment of input values and output value for every gate in C (i.e. the values of the wires)

- We check in polynomial (linear in # of gates) time if, for every gate, the computed output value matches the value of the output wire given in the certificate and the output of the whole circuit is 1, then the algorithm outputs 1, otherwise 0



Showing that CIRCUIT-SAT is NP-hard

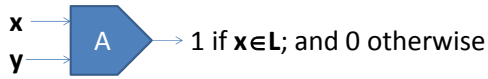
Main idea:

- For <u>**any**</u> problem **L∈NP**, show that **L ≤$_p$ CIRCUIT-SAT**
- In another word, we need a polynomial time program **f** that maps any instance **x∈L** to a circuit **C=f(x)** such that **x∈L** if and only if **C=f(x)** is satisfiable

A verifier program **A** for the problem **L**

Since **L∈NP**, there exists a program **A** that verifies if **x∈L** given a certificate **y**

- Precisely, for any instance **x∈L** and the corresponding certificate **y**, **A(x, y) = 1** if **x∈L**



We build <u>a circuit</u> to simulate the execution of the computer program **A**

- We can always do so, this circuit is our today's computer

Simulate program **A** as a circuit

The computation of **A** is viewed as a sequence of configurations: $c_0, c_1, \ldots, c_i, c_{i+1}, \ldots, c_{T(n)}$ and each c_i has:

- Program **A**, program counter **PC**, auxiliary machine state, the input ***x***, the certificate ***y***, working storage
- c_i is mapped to c_{i+1} by the combinational circuit **M** that implements the computer hardware
- The output of **A** (which is 0 or 1) is written to some designated location in working storage
- If the algorithm **A** runs for at most **T(n)** steps, then the output of algorithm **A** appears as one bit in configuration $c_{T(n)}$
  - Note: **A(x, y) = 1** or **0**
- Note: Visual explanation in the **<u>NEXT slide</u>**

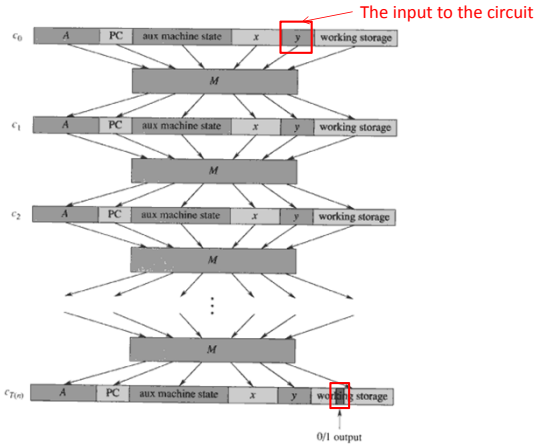


Figure 34.9 The sequence of configurations produced by an algorithm A running on an input x and certificate y. Each configuration represents the state of the computer for one step of the computation and, besides A, x, and y, includes the program counter (PC), auxiliary machine state, and working storage. Except for the certificate y, the initial configuration c_0 is constant. Each configuration is mapped to the next configuration by a boolean combinational circuit M. The output is a distinguished bit in the working storage.

Circuit **C=f(x)** is of polynomial size and can be constructed in polynomial time

For each c_i: **A** is a program that has a constant size (in binary); **PC**, **aux machine state, and working storage** have constant size; **x** is of length **n**; the certificate **y** is of length $O(n^k)$, for some constant **k≥1**

The verification algorithm **A** runs in **T(n)** steps

- **A** is a polynomial verification algorithm for problem **L∈NP**

The sub-circuit **M** between c_i and c_{i+1} is of polynomial size

Hence, the whole circuit is of polynomial size (at most $O(n^k)$ copies of **M**) and can be constructed in polynomial time

$x \in L \rightarrow C = f(x)$ is satisfiable

If $x \in L$, there exists a certificate **y** such that **A(x, y) = 1**

Then, with input **y**, the circuit **C = f(x)** is satisfiable

Recall that circuit **C = f(x)** is a *simulation of verification program **A***, it will give **A(x, y) = 1** if given certificate **y**

$x \in L \leftarrow C = f(x)$ is satisfiable

If **C = f(x)** is satisfiable, there exists some **y** such that the circuit **f(x)** is satisfiable

This implies that **A(x, y) = 1** because **C = f(x)** is the *simulation of verification program **A***

Thus, $x \in L$

CIRCUIT-SAT is NP-complete

In summary:

- CIRCUIT-SAT is NP as it is verifiable in polynomial time
- CIRCUIT-SAT is NP-hard as **every** NP problem can be reduced to CIRCUIT-SAT in polynomial time
  - We have shown that for **any** problem $L \in NP$, we can reduce it's polynomial time verifier algorithm **A** into a **CIRCUIT-SAT** instance in polynomial time
- Thus CIRCUIT-SAT is NP-complete

More Problems in **NPC**

Opening the Floodgates…

Opening the Floodgates

Now, we know that CIRCUIT-SAT is NPC,
can we find more NPC problems?

- Based on **polynomial time reduction**,
  we can generate **more** NPC problems

We will learn various reduction techniques
and by enlarging our library of NPC problems,
we may find easier polynomial time reduction strategy
for future NPC problems

NP-completeness proof basis

Lemma: Given a problem **L**, if **L'** $\leq_p$ **L** for some **L'**$\in$**NPC**, then **L** is **NP-hard**. Moreover, if **L**$\in$**NP**, then **L**$\in$**NPC**

Proof:

- Since **L'**$\in$**NPC**, we have **L''**$\leq_p$**L'** for all **L''**$\in$**NP**
- Hence, we have **L''**$\leq_p$**L'**$\leq_p$**L** for all **L''**$\in$**NP**
- Thus, **L** is **NP-hard**

If, in addition, **L**$\in$**NP**, then **L**$\in$**NPC**

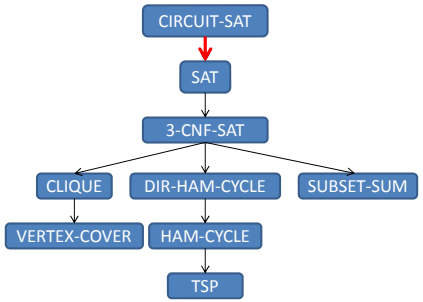
2 steps to prove L is NP-complete

1. Prove that **L**$\in$**NP**
   - Given a certificate, we prove that the certificate can be verified in polynomial time
2. Prove that **L** is **NP-hard**
   - Select a known **NP-complete** problem **L'** (important sub-step)
   - Show that **L'** $\leq_p$ **L**
     1. Identify a function ***f*** that maps <u>every instance</u> ***x*** of **L'** into an instance ***f*(*x*)** of **L**
     2. Prove ***f*** satisfies that, for all instance ***x*$\in$L'**, ***x*$\in$L'** is YES if and only if ***f*(*x*)$\in$L** is YES
     3. Prove that the algorithm that computes ***f*** runs in polynomial-time

This is a shortcut to show any (L''$\in$NP $\leq_p$ L') $\leq_p$ L
Because L' has been proven to be NP-complete

Agenda for showing more NP-complete problems

Using polynomial time reduction,
we can obtain **more** NP-complete problems



Satisfiability problem (SAT)

SAT has the historical honor to be the *first known* NP-complete problem (Cook-Levin theorem)

Input:

- ***n*** boolean variables: $x_1, x_2, \ldots, x_n$
- A boolean formula ϕ formed by the Boolean variables, the operators, such as $\wedge$ (and), $\vee$ (or), $\neg$ (not), $\rightarrow$ (implication), $\leftrightarrow$ (equality/biconditional), …, and parentheses

A SAT ϕ is satisfiable if there exists a truth assignment which causes ϕ to evaluate to 1

Output: "Yes" if ϕ is satisfiable; or "No" otherwise

Examples

Example 1: $\phi = x_1 \wedge x_2 \wedge x_3 \wedge (x_1 \vee \neg x_2) \wedge (x_2 \vee \neg x_3)$

- It is satisfiable by setting $x_1=1$, $x_2=1$, $x_3=1$

Example 2: $\phi = (x_1 \leftrightarrow x_2) \wedge (x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2)$

- It is not satisfiable

1. Does SAT belong to NP?

Yes, simple proof:

- Given a satisfying assignment, the verifying algorithm simply replaces each variable with the value as specified in the satisfying assignment, evaluates the formula in polynomial time, and check whether it outputs a 1

Hence, SAT is in NP

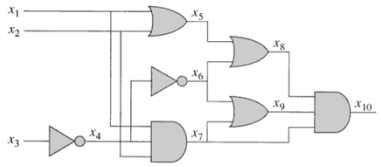
2. Is SAT an NP-hard problem?

We have to show that **CIRCUIT-SAT**$\leq_p$**SAT**, i.e. any instance of circuit satisfiability can be reduced in polynomial time to an instance of formula satisfiability

- *PS: We have no choice as we have only proven the NP-completeness of CIRCUIT-SAT so far*
- Given a circuit **N**, we want to form a Boolean formula **f(N)** such that **N** is satisfiable iff **f(N)** is satisfiable
- We also need to ensure that **f(N)** can be computed in polynomial time

Incorrect (non polynomial) reduction

Given a circuit, recursively expand the Boolean formula in a top-to-down fashion…



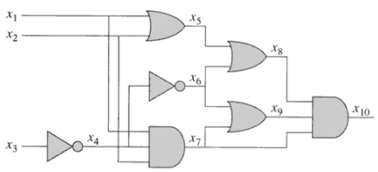
e.g. $\phi = x_{10} = x_7 \wedge x_8 \wedge x_9 = (x_1 \wedge x_2 \wedge x_4) \wedge (x_5 \vee x_6) \wedge (x_6 \vee x_7)$
$= (x_1 \wedge x_2 \wedge x_4) \wedge ((x_1 \vee x_2) \vee \neg x_4) \wedge (\neg x_4 \vee (x_1 \wedge x_2 \wedge x_4)) = \ldots$

- This transformation may generate exponential number of variables (Why?)

Correct polynomial reduction (1)

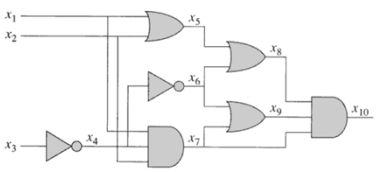
Given a circuit **N**, for each gate **G** with input $\mathbf{y_1, y_2, \ldots, y_k}$ and output **z**, we represent it as $\mathbf{z \leftrightarrow (y_1\ op\ y_2\ op \ldots op\ y_k)}$ where **op** depends on the gate **G** ($\leftrightarrow$ is logical equality)

We then AND the circuit output variable with the conjunction of clauses describing the operation of each gate



$$\begin{aligned}\phi = x_{10} \wedge (&x_{10} \leftrightarrow (x_7 \wedge x_8 \wedge x_9))\\ &\wedge (x_9 \leftrightarrow (x_6 \vee x_7))\\ &\wedge (x_8 \leftrightarrow (x_5 \vee x_6))\\ &\wedge (x_7 \leftrightarrow (x_1 \wedge x_2 \wedge x_4))\\ &\wedge (x_6 \leftrightarrow \neg x_4)\\ &\wedge (x_5 \leftrightarrow (x_1 \vee x_2))\\ &\wedge (x_4 \leftrightarrow \neg x_3)\end{aligned}$$

Correct polynomial reduction (2)



$$\begin{aligned}\phi = x_{10} \wedge (&x_{10} \leftrightarrow (x_7 \wedge x_8 \wedge x_9))\\ &\wedge (x_9 \leftrightarrow (x_6 \vee x_7))\\ &\wedge (x_8 \leftrightarrow (x_5 \vee x_6))\\ &\wedge (x_7 \leftrightarrow (x_1 \wedge x_2 \wedge x_4))\\ &\wedge (x_6 \leftrightarrow \neg x_4)\\ &\wedge (x_5 \leftrightarrow (x_1 \vee x_2))\\ &\wedge (x_4 \leftrightarrow \neg x_3)\end{aligned}$$

Note:

- **ϕ=f(N)** can be computed in polynomial time
- **N** is satisfiable iff **ϕ=f(N)** is satisfiable

Hence, CIRCUIT-SAT $\leq_p$ SAT

SAT is NP-complete

Since

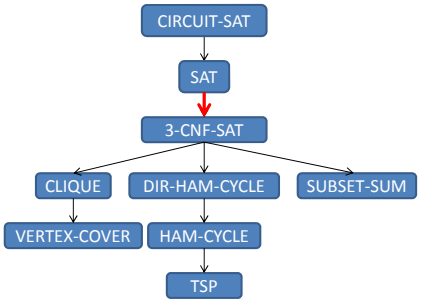
1. SAT is in NP; and
2. CIRCUIT-SAT (which is an NPC problem) $\leq_p$ SAT

We conclude that SAT is NPC

As of now, we know *at least two* NPC problems exists in the world: CIRCUIT-SAT and SAT, are there more?

Agenda for showing more NP-complete problems

Using polynomial time reduction,
we can obtain **more** NP-complete problems



3-CNF-SAT(isfiability)

- A literal in a Boolean formula is an occurrence of a variable or its negation
- CNF (Conjunctive Normal Form) is a Boolean formula expressed as <u>AND of clauses</u>, where each clause is the OR of one or more literals
- 3-CNF is a CNF in which each clause has <u>exactly 3 distinct literals</u> (a literal and its negation are distinct)
- **3-CNF-SAT** problem is therefore as follows:
  - **Input:** A 3-CNF ϕ
  - **Output:** Whether the 3-CNF ϕ is satiafiable?

Examples

3 literals, 2 ORs — AND — 3 literals, 2 ORs

Example 1: $\phi = (x \vee y \vee z) \wedge (x \vee y \vee \neg z)$

- Answer: Yes (certificate: x=1, y=0, z=0)

Example 2: $\phi = (x \vee y \vee z) \wedge (x \vee y \vee \neg z) \wedge (x \vee \neg y \vee z) \wedge (x \vee \neg y \vee \neg z) \wedge (\neg x \vee y \vee z) \wedge (\neg x \vee y \vee \neg z) \wedge (\neg x \vee \neg y \vee z) \wedge (\neg x \vee \neg y \vee \neg z)$

- Answer: No

This **3-CNF-SAT** problem is more convenient to handle than the general **SAT** (and also **CIRCUIT-SAT**) problem

1. 3-CNF-SAT$\in$NP

The certificate is the assignment of 0 and 1 for the variables, then we can evaluate in polynomial time if the 3-CNF boolean formula is satisifiable

Example: Given the certificate x=1, y=0, z=0,
we can show that $\phi = (x \vee y \vee z) \wedge (x \vee y \vee \neg z) = 1$

Hence, 3-CNF-SAT$\in$NP

2. **SAT$\leq_p$3-CNF-SAT**

Given a Boolean formula ϕ, we will give a two-step algorithm to obtain a 3-CNF $\phi''=f_2(f_1(\phi))$

Step 1: $\phi'=f_1(\phi)$ generates a Boolean formula expressed as AND of clauses where each clause has 3 literals

- ϕ' is satisfiable iff ϕ is satisfiable

Step 2: $\phi''=f_2(\phi')$ converts every clause in ϕ' into 3-CNF

- ϕ'' is satisfiable iff ϕ' is satisfiable

We show that the conversion takes polynomial time

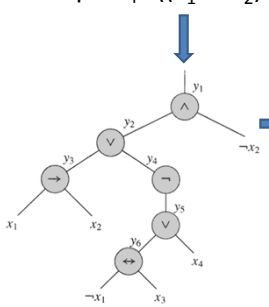
$f_1(\phi)$ (description)

1. We construct a binary 'parse' tree ('circuit') of ϕ, literals as leaves and connectives as internal vertices
2. We introduce a variable y_i for the output of each internal vertices
3. We rewrite the formula to ϕ' as the AND of the root variable and a conjunction of clauses describing the operation of each vertex (similar as the previous **CIRCUIT-SAT$\leq_p$SAT** reduction)

Therefore in ϕ', each clause has at most three literals

$f_1(\phi)$ (visual example)

Example: $\phi=((x_1\rightarrow x_2) \vee\neg((\neg x_1\leftrightarrow x_3) \vee x_4))\wedge\neg x_2$



$$\begin{aligned}\phi' = f_1(\phi) = {} & y_1\wedge(y_1 \leftrightarrow (y_2\wedge\neg x_2)) \\ & \wedge(y_2 \leftrightarrow (y_3 \vee y_4)) \\ & \wedge(y_4 \leftrightarrow \neg y_5) \\ & \wedge(y_3 \leftrightarrow (x_1 \rightarrow x_2)) \\ & \wedge(y_5 \leftrightarrow (y_6 \vee x_4)) \\ & \wedge(y_6 \leftrightarrow (\neg x_1 \leftrightarrow x_3))\end{aligned}$$

$f_2(\phi')$ (description)

We change each clause ϕ_i' into CNF as follows:

1. Construct a truth table for ϕ_i'
   - This table will be small, at most 2^3=8 rows by 4 columns
2. Write the Disjunctive Normal Form (DNF) for all truth table items that evaluate to 0
   - We will later negate this back using DeMorgan's law (step 3), that's why we need items that evaluate to 0 for now
3. By using DeMorgan's law, we obtain the CNF
   - $\neg(a\wedge b) = \neg a\vee \neg b$
   - $\neg(a\vee b) = \neg a\wedge\neg b$

$f_2(\phi')$ (example)

Example: Converting $\phi_2' = y_1 \leftrightarrow (y_2 \wedge \neg x_2)$ into 3-CNF

- This is the second clause of our example ϕ' two slides ago

y_1	y_2	x_2	$(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	1
0	0	0	1

Disjunctive Normal Form:

$\phi_2'=(y_1 \wedge y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge x_2)$
$\vee (y_1 \wedge \neg y_2 \wedge \neg x_2) \vee (\neg y_1 \wedge y_2 \wedge \neg x_2)$

Conjunctive Normal Form:

$\phi_2''=(\neg y_1 \vee \neg y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee \neg x_2)$
$\wedge (\neg y_1 \vee y_2 \vee x_2) \wedge (y_1 \vee \neg y_2 \vee x_2)$

$f_2(\phi')$ (final cleanup)

The resulting clauses are in CNF and each clause has 3 <u>(or less)</u> literals

For the final step, we change 1 or 2-literal clause into 3-literal clause as follows:

- If a clause has one literal l,
  change it to $(l \vee p \vee q) \wedge (l \vee p \vee \neg q) \wedge (l \vee \neg p \vee q) \wedge (l \vee \neg p \vee \neg q)$
- If a clause has two literals $(l_1 \vee l_2)$,
  change it to $(l_1 \vee l_2 \vee p) \wedge (l_1 \vee l_2 \vee \neg p)$

3-CNF-SAT is NP-complete

Given a Boolean formula ϕ, we have shown that we can reduce it to a 3-CNF formula ϕ'' by (1) $\phi'=f_1(\phi)$ and then (2) $\phi''=f_2(\phi')$

Note that:

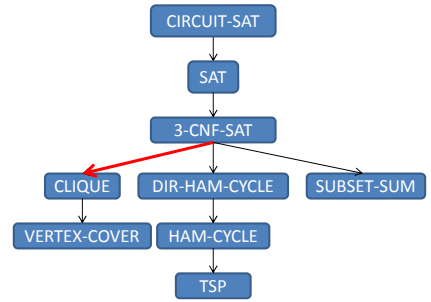
- ϕ is satisfible iff ϕ' is satisfible iff ϕ'' is satisfible
- Also, the reduction takes polynomial time

Hence, 3-CNF-SAT is NP-hard

As 3-CNF-SAT is in NP, 3-CNF-SAT is NP-complete

Agenda for showing more NP-complete problems

Using polynomial time reduction,
we can obtain **more** NP-complete problems



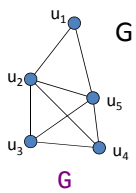
CLIQUE: From Satisfiability to Graph

Input: An undirected graph **G=(V,E)** and a constant **k**

- Definition: A clique is a complete subgraph of **G**, i.e., a subset **V'⊆V** such that **(u, v)∈E** for every **u,v∈V'**

Output: Whether **G** contains a clique of size **k**

Example:



G contains a size-4 clique $\{u_2,u_3,u_4,u_5\}$

1. CLIQUE∈NP

Input: G=(V,E) and **k**

Certificate: A size-**k** subset **V'⊆V**

The verifying algorithm simply check for each pair of **u,v∈V'** whether **(u,v)∈E**

Running time: **O(|V'|²|E|)**, polynomial time

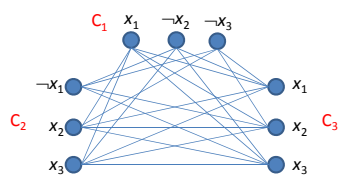
- Q: Design a *better* polynomial time verifying algorithm?

2. 3-CNF-SAT$\leq_p$CLIQUE

Given a 3-CNF $\phi=C_1\wedge C_2\wedge\ldots\wedge C_k$ formula, we construct a graph **G=(V,E)**

- **V** is the set of all literals of all clauses, i.e. we have **3k** vertices
- For any two literals **a** and **b**, **(a,b)∈E** if (1) they are in different clauses and (2) **a** is not the negation of **b**

Example $\phi=C_1\wedge C_2\wedge C_3$ where
$C_1=(x_1\vee\neg x_2\vee\neg x_3)$,
$C_2=(\neg x_1\vee x_2\vee x_3)$, and
$C_3=(x_1\vee x_2\vee x_3)$



ϕ is satisfiable → G has a size-k clique

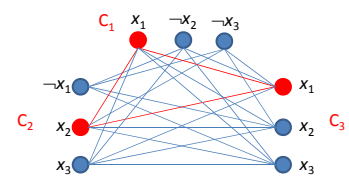
If ϕ is satisfiable, then there is a satisfying assignment

For each clause C_i, there must be a literal v_i that evaluates to 1

By definition, $\{v_1, v_2, \ldots, v_k\}$ form a clique

Example: For $\phi=(x_1\vee\neg x_2\vee\neg x_3)\wedge(\neg x_1\vee x_2\vee x_3)\wedge(x_1\vee x_2\vee x_3)$

- A satisfying assignment is $x_1=1$, $x_2=1$, $x_3=1$
- For each clause, we select a literal that evaluates to 1
  - Say, we select x_1 in C_1, x_2 in C_2, and x_1 in C_3, notice that they form a clique of size 3



ϕ is satisfiable ← G has a size-k clique

Suppose G has a clique V' of size k

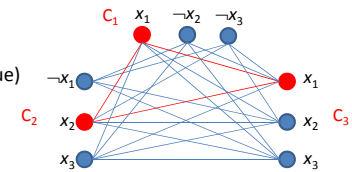
Since no edge is allowed between literals in the same clause, each clause has exactly one literal belonging to V'

Assign all literals correspond to the vertices in V' to 1 and all other literals arbitrarily (for simplicity, let's assign them to 0), then, every clause evaluate to 1

So, ϕ is satisfiable

Example:

- We have a 3-clique in G (not unique)
- Then, we set $x_1=x_2=1$, $x_3=0$
- Then, ϕ is evaluated to 1



CLIQUE is NP-complete

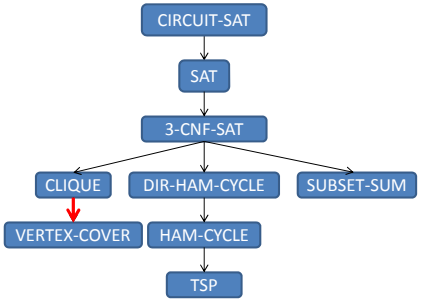
We have shown that:

1. CLIQUE is in NP
2. CLIQUE is NP-Hard

Therefore, CLIQUE is NP-Complete

Agenda for showing more NP-complete problems

Using polynomial time reduction, we can obtain **more** NP-complete problems

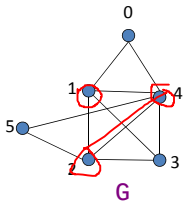


VERTEX-COVER

Input: An undirected graph **G = (V,E)** and an integer **k**

Definition: **V'⊆V** is a vertex cover of **G** if for all **(u,v)∈E**, **u∈V'** or **v∈V'** (or both)

Output: Decide if **G** contains a vertex cover of size **k**



{1, 2, 4} is a vertex cover of G with size 3

1. VERTEX-COVER∈NP

Input: An undirected graph **G = (V,E)** and an integer **k**

Certificate: A subset **V'** of size **k**

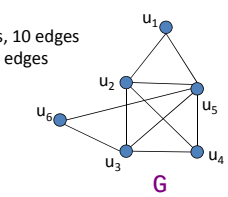
The verifying algorithm checks if **|V'|=k** in **O(V)** and then scans all edge **(u,v)∈E** in **O(E)** to check if at least one of **u** and **v** belongs to **V'**

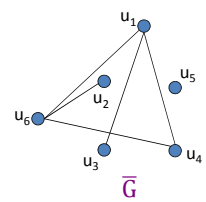
Therefore, **Vertex Cover is in NP**

2. CLIQUE$\leq_p$VERTEX-COVER

Given an undirected graph **G=(V,E)** and **k** (**|V|=n**), we construct a graph $\bar{G}$**=(V,Ē)** where **(u,v)∈Ē** iff **(u,v)∉E**

6 vertices, 10 edges
K_6 has 15 edges





6 vertices, 5 edges

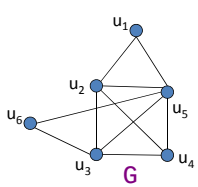
Claim: G has a size-**k** clique iff $\bar{G}$ has a size-**(n-k)** vertex cover

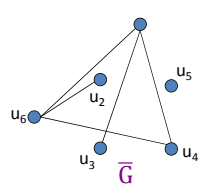
G has a size-k clique → $\bar{G}$ has a size-(n-k) vertex cover

Suppose V' is size-k clique of G, therefore for every edge (u,v)∈Ē, it is not possible that both u and v ∈ V' as every pair of vertices in V' are connected (remember that it is a clique)

Hence, at least one of u and v belongs to V-V' and thus, V-V' (of size n-k) is a vertex cover of $\bar{G}$

Example: $\{u_2,u_3,u_4,u_5\}$ is a size-4 clique of G → $\{u_1,u_6\}$ is a size-2 vertex cover of $\bar{G}$





G has a size-k clique ← $\bar{G}$ has a size-(n-k) vertex cover

Let U be the size-(n-k) vertex cover of $\bar{G}$

- By definition, for all u,v∈V, if (u,v)∈Ē, then at least one of u and v belong to U
- The contrapositive of this statement is for all u,v∈V and both u and v do not belong to U → (u,v)∉Ē → (u,v)∈E

Hence, V-U is a clique and has size = **n-(n-k) = k**

VERTEX-COVER is NP-Complete

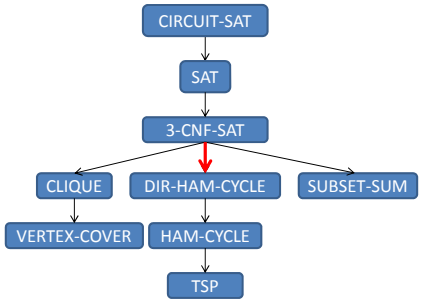
We have shown that:

1. VERTEX-COVER is in NP
2. VERTEX-COVER is NP-Hard

Therefore, VERTEX-COVER is NP-Complete

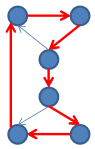
Agenda for showing more NP-complete problems

Using polynomial time reduction,
we can obtain **more** NP-complete problems

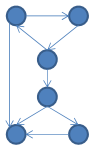


Directed Hamiltonian Cycle

- DIR-HAM-CYCLE: Given a digraph G = (V, E), does there exists a simple directed cycle Γ that contains every node in V exactly once?



Has Hamiltonian cycle



No Hamiltonian cycle

1. **DIR-HAM-CYCLE∈NP**

Given a directed graph **G=(V,E)** and a certificate:
A cycle C=(v_1, v_2, …, v_n, v_1), the verifying algorithm checks if:

1. C has each vertex in V exactly once
2. The first and the last vertex in C is the same (a cycle)
3. All consecutive vertices (v_1, v_2), (v_2, v_3), …, (v_{n-1}, v_n), (v_n, v_1) $\in$ E

This check can be done in O(V+E), i.e. polynomial time

Hence, **DIR-HAM-CYCLE∈NP**

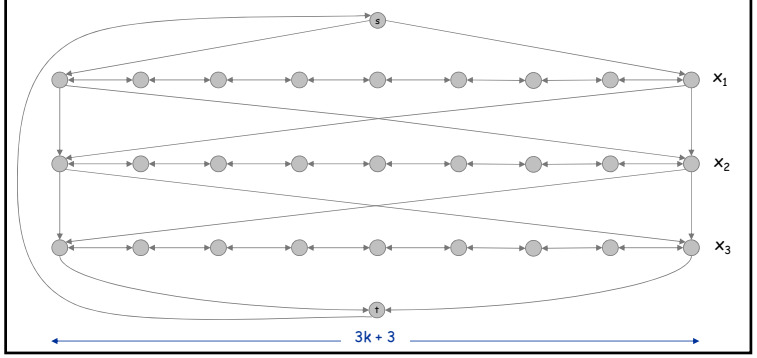
2. 3-CNF-SAT $\leq_p$ DIR-HAM-CYCLE

- Claim: 3-CNF-SAT $\leq_p$ DIR-HAM-CYCLE.
- Proof: Given an instance Φ of 3-CNF-SAT, we construct an instance of DIR-HAM-CYCLE that has a Hamiltonian cycle iff Φ is satisfiable.
- Construction. First, create a graph that has 2^n Hamiltonian cycles which correspond in a natural way to 2^n possible truth assignments.

3-CNF-SAT Reduces to DIR-HAM-CYCLE

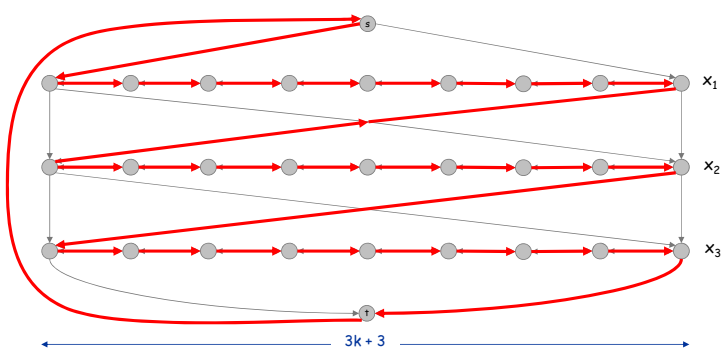
Construction. Given 3-SAT instance Φ with n variables x_i and k clauses.

- Construct G to have 2^n Hamiltonian cycles.
- Intuition: traverse path i from left to right $\Leftrightarrow$ set variable $x_i = 1$.



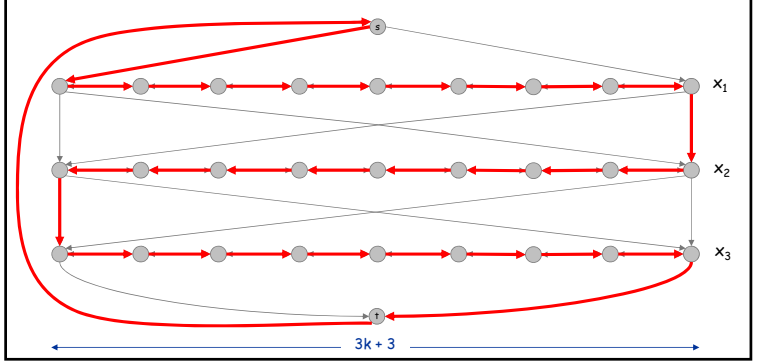
3-CNF-SAT Reduces to DIR-HAM-CYCLE

Example: $x_1=1$, $x_2=1$, $x_3=1$



3-CNF-SAT Reduces to DIR-HAM-CYCLE

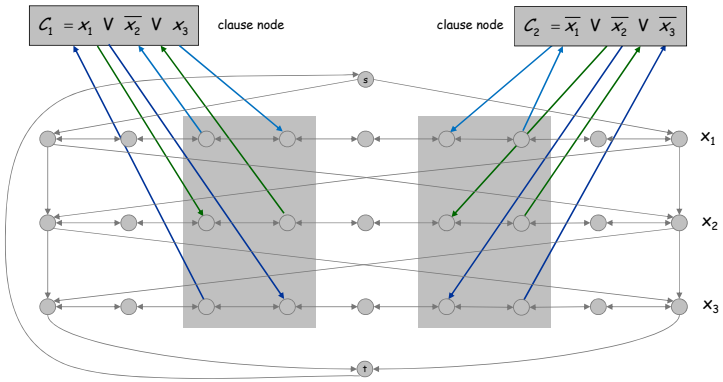
Another example: $x_1=1$, $x_2=0$, $x_3=1$



3-CNF-SAT Reduces to DIR-HAM-CYCLE

Construction. Given 3-CNF-SAT instance Φ with n variables x_i and k clauses.

- For each clause: add a node and 6 edges.



3-CNF-SAT Reduces to DIR-HAM-CYCLE

Claim. Φ is satisfiable iff G has a Hamiltonian cycle.

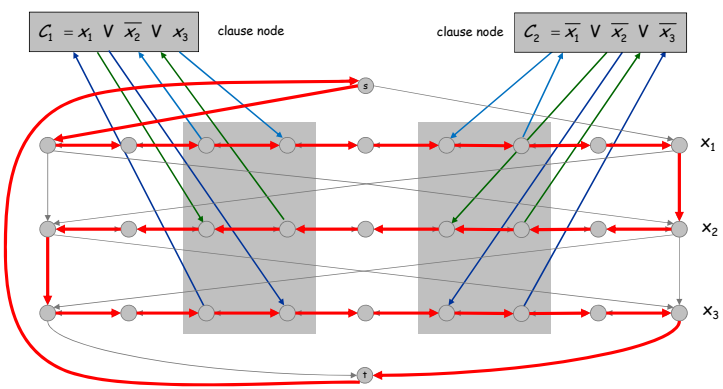
Pf. $\Rightarrow$

- Suppose 3-CNF-SAT instance has satisfying assignment x*.
- Then, define Hamiltonian cycle in G as follows:
  - if $x^*_i = 1$, traverse row i from left to right
  - if $x^*_i = 0$, traverse row i from right to left
  - for each clause C_j, there will be at least one row i in which we are going in "correct" direction to splice node C_j into tour

66

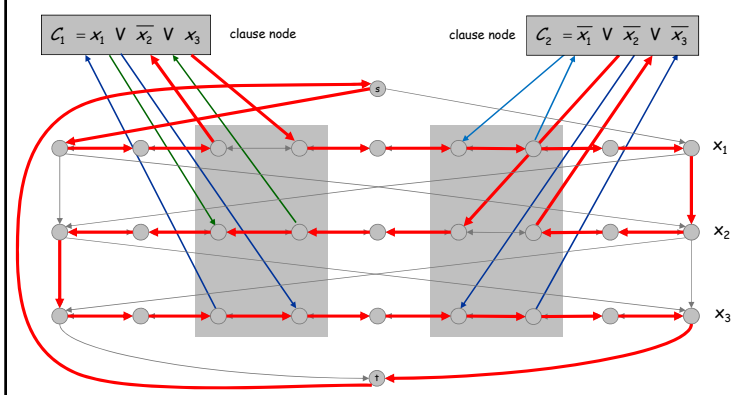
3-CNF-SAT Reduces to DIR-HAM-CYCLE

Example: x_1=1, x_2=0, x_3=1 satisfy C_1 and C_2. The corresponding cycle is as follows.



3-CNF-SAT Reduces to DIR-HAM-CYCLE

Example: x_1=1, x_2=0, x_3=1 satisfy C_1 and C_2. The corresponding cycle is as follows. We can amend the cycle to also visit the two clause nodes.



3-CNF-SAT Reduces to DIR-HAM-CYCLE

Claim. Φ is satisfiable iff G has a Hamiltonian cycle.

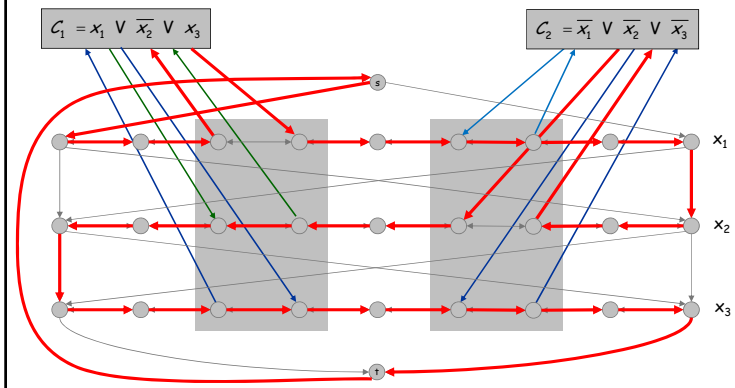
Pf. $\Leftarrow$

- Suppose G has a Hamiltonian cycle Γ.
- For each clause node C_j,
  - Find the edge in Γ that goes in C_j and the edge that goes out.
  - Note that these two edges must be mate pair (otherwise, some node is never visited).
  - thus, nodes immediately before and after C_j are connected by an edge e_j in G
  - removing C_j from cycle, and replacing it with edge e_j yields Hamiltonian cycle on $G - \{ C_j \}$
- Set $x^*_i = 1$ iff Γ' traverses row i left to right.
- For each clause C_j, it is satisfied since the edge e_j has a "correct" direction.

69

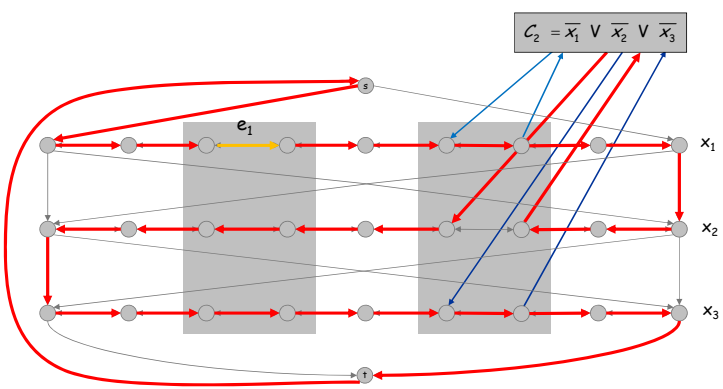
3-CNF-SAT Reduces to DIR-HAM-CYCLE

Consider the following Hamiltonian cycle Γ.
Remove clause node C_1.



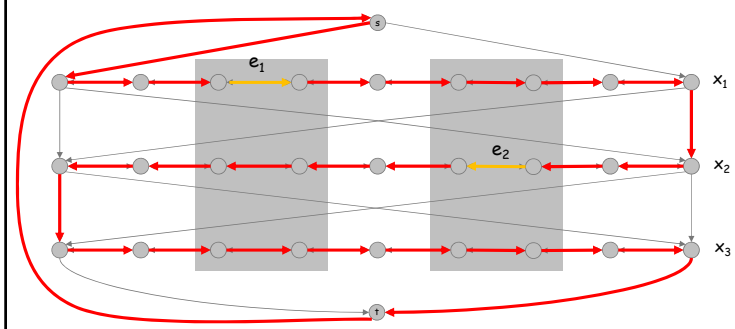
3-CNF-SAT Reduces to DIR-HAM-CYCLE

Remove clause node C_2.



3-CNF-SAT Reduces to DIR-HAM-CYCLE

Determine x_1=1, x_2=0 and x_3=1.
From e_1, clause $C_1 = x_1 \vee \overline{x_2} \vee x_3$ is true since x_1=1.
From e_2, clause $C_2 = \overline{x_1} \vee \overline{x_2} \vee \overline{x_3}$ is true since x_2=0.



DIR-HAM-CYCLE is NP-complete

We have shown that:

1. DIR-HAM-CYCLE is in NP
2. DIR-HAM-CYCLE is NP-Hard

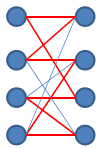
Therefore, DIR-HAM-CYCLE is NP-Complete

HAM(iltonian)-CYCLE

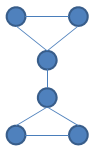
Input: An undirected graph **G=(V,E)**

- Definition: A Hamiltonian cycle is a simple cycle that contains <u>all vertices</u> in **V**

Output: Decide if G contains a Hamiltonian cycle



The red color cycle is a Hamiltonian cycle



No Hamiltonian cycle

1. **HAM-CYCLE$\in$NP**

Given an undirected graph **G=(V,E)** and a certificate: A cycle $C=(v_1, v_2, \ldots, v_n, v_1)$, the verifying algorithm checks if:

1. C has each vertex in V exactly once
2. The first and the last vertex in C is the same (a cycle)
3. All consecutive vertices $(v_1, v_2), (v_2, v_3), \ldots, (v_{n-1}, v_n), (v_n, v_1) \in E$

This check can be done in O(V+E), i.e. polynomial time

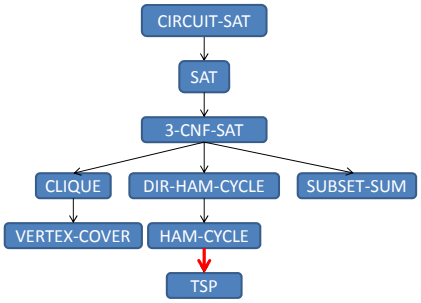
Hence, **HAM-CYCLE$\in$NP**

2. **DIR-HAM-CYCLE$\leq_p$HAM-CYCLE**

Exercise!

Agenda for showing more NP-complete problems

Using polynomial time reduction,
we can obtain **more** NP-complete problems



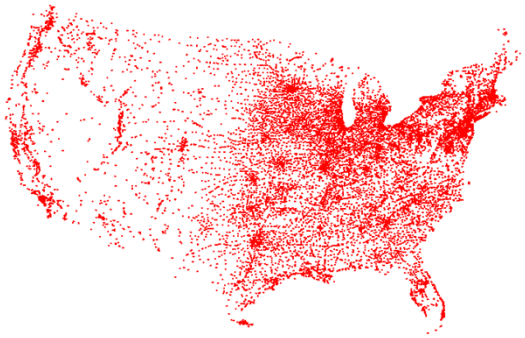
Traveling-Salesman Problem (TSP)

Input:

- A complete directed graph **G=(V, VxV)**
- A cost function **c: VxV→Z**
- A cost threshold **k**

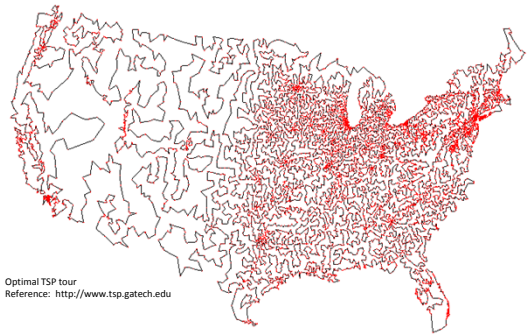
Output: Decide if there exists a tour that visits all vertices in **V** with cost at most **k**

TSP Example (1)



All 13,509 cities in US with a population of at least 500
Reference: http://www.tsp.gatech.edu

TSP Example (2)



Optimal TSP tour
Reference: http://www.tsp.gatech.edu

Solved on May 8, 1998. Running time: 131,052,087 seconds (about 4.16 years)

There is another USA challenge with 115,475 cities
But no one know what is the optimal solution yet…

1. **TSP$\in$NP**

Given a certificate**:** The tour T: $(v_1, v_2, \ldots, v_n, v_1)$,
the verifying algorithm checks if:

1. T has each vertex in V exactly once
2. The first and the last vertex in T is the same (a cycle)

Note: As the input is a complete graph, we do not need to check if all consecutive vertices $(v_1, v_2), \ldots, (v_{n-1}, v_n), (v_n, v_1) \in E$, it will be always true

3. $c(v_n, v_1) + \sum_{i=1..n-1} c(v_i, v_{i+1}) \leq k$

This is doable in linear (polynomial) time,
Hence, **TSP$\in$NP**

2. **HAM-CYCLE$\leq_p$TSP**

Transformation:

- Input: G=(V, E)
- Set H=(V, VxV)
  - c(v,u) = c(u,v) = 0 if (u,v)$\in$E; and 1 otherwise
  - Set k = 0

Claim: G has a Hamiltonian cycle iff
H has a tour with cost 0

G has a Hamiltonian cycle → H has a tour with cost 0

Let $(v_1, v_2, \ldots, v_n, v_1)$ be the Hamiltonian cycle in G

Then, $(v_1, v_2, \ldots, v_n, v_1)$ is a cost-0 tour in H

- Reason: $(v_i, v_{i+1}) \in E \rightarrow c(v_i, v_{i+1})=0$ for i=1, 2, …, n-1
- Also, the last edge $(v_n, v_1) \in E \rightarrow c(v_n, v_1)=0$

G has a Hamiltonian cycle ← H has a tour with cost 0

Let $(v_1, v_2, \ldots, v_n, v_1)$ be a cost-0 tour in H

Then, $(v_1, v_2, \ldots, v_n, v_1)$ is a Hamiltonian cycle in G

- Reason:
  - Since the cost of the tour is 0,
    we have $c(v_n, v_1)=0$ and $c(v_i, v_{i+1})=0$ for i=1, 2, …, n-1
  - $c(v_i, v_{i+1})=0 \rightarrow (v_i, v_{i+1}) \in E$ for i=1, 2, …, n-1
  - Also, $c(v_n, v_1)=0 \rightarrow (v_n, v_1) \in E$
  - Hence, $(v_1, v_2, \ldots, v_n, v_1)$ is a Hamiltonian cycle in G

TSP is NP-complete

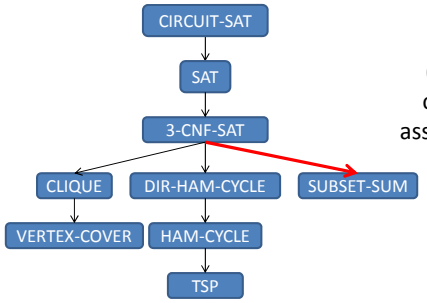
We have shown that:

1. TSP is in NP
2. TSP is NP-Hard

Therefore, TSP is NP-Complete

Agenda for showing more NP-complete problems

Using polynomial time reduction,
we can obtain **more** NP-complete problems



(Complex) proof omitted, you can assume SUBSET-SUM is indeed NP-complete

Our Current Library of NPC Problems

We have seen that quite a lot of problems are NPC:

1. CIRCUIT-SAT
2. SAT
3. 3-CNF-SAT
4. CLIQUE
5. VERTEX-COVER
6. DIR-HAM-CYCLE
7. HAM-CYCLE (detailed proof omitted)
8. TSP
9. SUBSET-SUM (detailed proof omitted)

We can use <u>any of them</u> as starting point to prove that some <u>other problems</u> are NPC too

Hopefully you are not surprised with this statement:
There are <u>many more</u> NPC problems beyond this lecture
e.g. see: http://en.wikipedia.org/wiki/List_of_NP-complete_problems

Final Slide: What to do next? :O

Knowing that <u>your problem</u> is NPC, should you stop?

No!

- L9 → graph lecture, we give examples of <u>special cases</u> of NPC problems that are still solvable in polynomial time
- L10 → DP, some NPC problems have DP solution in pseudo polynomial or exponential time
- L11 → we discuss greedy algorithms
- L12 → we discuss approximation algorithms

Subset-Sum problem

- Subset-sum belongs to NP
  - Certificate: $S' \subseteq S$
  - Verification algorithm just sum the values in S' and check if the sum equals t.
  - Time: O(|S|).
  - Hence, Subset-sum is in NP.
- Subset-sum is NP-hard
  - 3-CNF-SAT$\leq_p$SUBSET-SUM

Transform 3-CNF-SAT to Subset-SUM

- Input: A 3-CNF formula $\phi = C_1 \wedge C_2 \wedge \ldots \wedge C_k$ with literals $x_1, x_2, \ldots, x_n$.
- Assumption: No clause contains both the literal and its negation.
- We construct a set S of 2(k+n) integers:
  - Each integer has k+n digits (base 10).
  - The first n digits represent the n literals while the remaining k digits represent the k clauses.
- For each C_j, create two integers:
  - $s_j = 00\ldots0\,|\,0\ldots01_{(j)}0\ldots0$
  - $s'_j = 00\ldots0\,|\,0\ldots02_{(j)}0\ldots0$
- For each literal x_i, create two integers:
  - $v_i = 0\ldots01_{(i)}0\ldots0 \mid 0\ldots01_{(l)}0\ldots01_{(w)}0\ldots0$, where x_i appears in $C_l,\ldots,C_w$.
  - $v'_i = 0\ldots01_{(i)}0\ldots0 \mid 0\ldots1_{(m)}0\ldots\ldots01_{(p)}0\ldots0$, where $\neg x_i$ appears in $C_m,\ldots,C_p$.
- Set t = 1…1 | 4…4.

Example

- $\phi = C_1 \wedge C_2 \wedge \ldots \wedge C_k$ with
- $C_1 = (x_1 \vee \neg x_2 \vee \neg x_3)$,
- $C_2 = (\neg x_1 \vee \neg x_2 \vee \neg x_3)$,
- $C_3 = (\neg x_1 \vee \neg x_2 \vee x_3)$ and
- $C_4 = (x_1 \vee x_2 \vee x_3)$.

	x_1	x_2	x_3	C_1	C_2	C_3	C_4
s_1	0	0	0	1	0	0	0
s'_1	0	0	0	2	0	0	0
s_2	0	0	0	0	1	0	0
s'_2	0	0	0	0	2	0	0
s_3	0	0	0	0	0	1	0
s'_3	0	0	0	0	0	2	0
s_4	0	0	0	0	0	0	1
s'_4	0	0	0	0	0	0	2
v_1	1	0	0	1	0	0	1
v'_1	1	0	0	0	1	1	0
v_2	0	1	0	0	0	0	1
v'_2	0	1	0	1	1	1	0
v_3	0	0	1	0	0	1	1
v'_3	0	0	1	1	1	0	0
t	1	1	1	4	4	4	4

Note:
For the first n positions, the sum of digits in each position is 2.
For the remaining k positions, the sum of digits in each position is 6.

ϕ is satisfible → there is subset S' whose sum is t

- Suppose ϕ has a satisfying assignment.
- For i=1,…,n,
  - if x_i=1 in the assignment, then v_i is put in S';
  - otherwise, v_i' is put in S'.
- The digits labeled by literals will sum to 1.
  0
- For j=1,…,k, s_j and/or s_j' are added to S' to make the sum of the digit to 4. Precisely,
  - if C_j has 1 literal assigned to 1, both s_j and s_j' are added to S'.
  - if C_j has 2 literal assigned to 1, s_j' is added to S'.
  - if C_j has 3 literal assigned to 1, s_j is added to S'.
- The digits labeled by clauses will sum to 4.
- So S' will sum to 1…14…4.

Example

- The satisfying assignment of ϕ:
- $x_1=0$, $x_2=0$ and $x_3=1$.

- Put v_1, v_2, v'_3 in S'.
- C_1 has 1 literal assigned 1, put s_1 and s'_1 in S'.
- C_2 has 2 literal assigned 1, put s'_2 in S'.
- C_3 has 3 literal assigned 1, put s_3 in S'.
- C_4 has 1 literal assigned 1, put s_4 and s'_4 in S'.

- Hence, $S'=\{v_1, v_2, v'_3, s_1, s'_1, s'_2, s_3, s_4, s'_4\}$

	x_1	x_2	x_3	C_1	C_2	C_3	C_4
s_1	0	0	0	1	0	0	0
s'_1	0	0	0	2	0	0	0
s_2	0	0	0	0	1	0	0
s'_2	0	0	0	0	2	0	0
s_3	0	0	0	0	0	1	0
s'_3	0	0	0	0	0	2	0
s_4	0	0	0	0	0	0	1
s'_4	0	0	0	0	0	0	2
v_1	1	0	0	1	0	0	1
v'_1	1	0	0	0	1	1	0
v_2	0	1	0	0	0	0	1
v'_2	0	1	0	1	1	1	0
v_3	0	0	1	0	0	1	1
v'_3	0	0	1	1	1	0	0
t	1	1	1	4	4	4	4

ϕ is satisfible $\leftarrow$ there is subset S' whose sum is t

- Suppose there is subset S' whose sum is t=1…14…4.

- For i=1,…,n, since the sum of the digits of column x_i in S' is 1, S' contains exactly one of v_i and v'_i.
  - if $v_i \in S'$, set $x_i=1$;
  - Otherwise, set $x_i=0$.

- Let R be $S' \cap \{v_i, v'_i: i=1,\ldots,n\}$
- For j=1,…,k, since the sum of the digits of column C_j in S' is 4,
  - the sum of the digits of column C_j in R is at least 1.
  - This implies every clause C_j evaluate to 1.

- Hence, ϕ is satisfible.